

ECE 50877 Introduction to Applied Cryptography Project Proposals

Your N. Here
Purdue University

Second Name
Purdue University

Third Name
Purdue University

Zeke Ulrich
Purdue University

September 10, 2026

1 Improving the Security Model of Desktop Linux Through Isolation of Users and Applications

This proposal originates with the authors of privsec.dev and is paraphrased from their article on insecurities in Linux.

Title: Improving the Security Model of Desktop Linux Through Isolation of Users and Applications

Problem Definition: Operating systems like Android and ChromeOS have full system mandatory access control; that is, every process from the init process is strictly confined. Regardless of which application you install or how you install them, they have to play by the rules of an untrusted SELinux domain and are only able to utilize unprivileged APIs.

On Linux, it is quite the opposite. Out of the box, most systems only have a few system daemons confined. Some Linux distributions don't even have a Mandatory Access Control system at all. Applications are designed in an environment where they expect to be able to do whatever they want, and the app sandboxes/mandatory access control system are merely an afterthought trying to restrict an app to only access what it expects to be accessible.

To make matters worse, some system daemons are not designed with permission control in mind at all. For example, PulseAudio does not have any concept of audio in or out permission. Thus, the user is often left with only the choice of granting an app access to the socket or not. If they want to block microphone access, they have to block access to the socket, and thus break audio playback in the process. If they do want an audio playback, then they have to allow access to the PulseAudio socket, which in turns give an app unrestricted access to record them at any moment.

The only way to systematically fix this problem is to design a whole new system from scratch with a permission model like that of Android in mind. And even when that happens, it will take substantial work to get developers to develop their apps for said system.

Proposed Methodology: We will evaluate isolation primitives available on modern Linux distributions and measure how effectively they can be composed into an Android-style permission model without a full OS rewrite. First, we will inventory the kernel and userspace mechanisms available for confinement: Linux namespaces (user, mount, network, PID), cgroups v2 for resource control, seccomp-bpf for syscall filtering, and the existing mandatory access control frameworks (SELinux, AppArmor). Second, we will build a reference sandbox for a small set of representative desktop applications (a web browser, a video conferencing client, and a PDF viewer) using a bubblewrap/Flatpak-style unprivileged namespace sandbox combined with a portal layer that mediates access to resources such as the filesystem, camera, and audio devices, modeled on the xdg-desktop-portal approach. Third, to address the PulseAudio permission gap described above, we will implement a proxy layer between applications and the audio server that exposes separate, individually revocable capabilities for playback and capture, rather than a single all-or-nothing socket grant. Fourth, we will define a threat model consisting of a malicious or compromised application attempting privilege escalation, unauthorized device access, and cross-application data exfiltration, and we will test our sandboxed applications against this threat model using a combination of manual penetration testing and automated fuzzing of the portal interfaces. Finally, we will benchmark the performance and compatibility overhead (application startup time, IPC latency, and the number of application features that break under confinement) introduced by the sandbox to assess practical deployability.

Expected Findings: We expect that namespaces, seccomp, and a portal-mediated permission layer can approximate a meaningful subset of the Android permission model on existing Linux distributions without requiring a new kernel or init system, though with measurable gaps compared to a system designed for confinement from the ground up. We anticipate that unmodified applications will require the most compatibility work around filesystem access patterns and legacy IPC mechanisms such as D-Bus and PulseAudio.

2 Differential Timing Analysis for Cryptographic Library Fingerprinting

This proposal originates with Dr. Zahra Ghodsi of Purdue University.

Title: Differential Timing Analysis for Cryptographic Library Fingerprinting

Problem Definition: The starting step of exploitation is determining the target’s technology stack. An important part of this process is identifying the cryptographic libraries and implementations running on the target. Existing tools can often identify software through explicit version information or protocol behavior, but such information may be unavailable or intentionally hidden. This motivates the following research question: Can differential timing analysis identify the cryptographic library and version used by a remote target, despite network and system noise?

Proposed Methodology: We will construct a testbed containing multiple implementations and versions of commonly used cryptographic libraries. We will first collect timing measurements while each implementation performs a controlled suite of cryptographic operations under different inputs and configurations. We will analyze the resulting timing distributions to identify features that distinguish implementations. We will then train and evaluate statistical and machine-learning classifiers using these timing features.

Expected Findings: We expect the number of samples required to train accurate classifiers to rise substationally with noise. We also expect machine learning classifiers to outperform statistical classifiers, indicating broader applicability of machine learning techniques to side channel analysis.